Email: John.dixon@hull.ac.uk
Meetings: https://tinyurl.com/jdixonhull
Feedback: https://tinyurl.com/jdhullfeedback
Office: RBB-315

Message-based patterns

Distributed Systems Programming

Dr John Dixon

UNIVERSITY of HULL

Overview

We are looking at these topics today:

- Architectural Pattern Types
- Message-based Patterns
- Messaging
- Pipelines
- Transformations

UNIVERSITY of HULL

Patterns

- A design pattern
  - *"… describes a problem which occurs over and over in our environment, and then describes the core of the solution to that problem, in such a way that you can use this solution a million times over, without ever doing it the same way twice."*

[Christopher Alexander – *The Timeless Way of Building - 1977*]

This book proposed a theory of architectural patterns (for buildings but also more generally for any design) which allow replication of quality through shared template solutions to common problems.

UNIVERSITY of HULL

Patterns

Often we use a pattern without realising.

- The best programmers know the patterns already and can choose a selection of patterns that make sense for any given scenario.
- The performance, scalability and stability of our systems are often affected by the pattern(s) we choose, as are the maintainability, readability and expandability.
- It is worth considering that frameworks also use patterns, so by understanding what's going on under the hood we can choose a framework that is right for our purposes.

UNIVERSITY of HULL

Pattern properties

- Context
  - Describes the situation in which the pattern must operate
  - Includes brief descriptions of relevant factors
- Problem
  - Describes the target functionality required
  - Should be as closely defined as possible
- Forces
  - Summary of the external influences on the solution design
  - Lists any pressures, constraints and desirable features of the potential solutions
  - May include conflicting requirements
- Solution
  - A description of the pattern which will solve the described problem
  - Level of detail will depend upon the type of pattern

UNIVERSITY of HULL

"Each pattern then depends both on the smaller patterns it contains, and on the larger patterns within which it is contained."

[Christopher Alexander]

Patterns

- Patterns exist at different levels of abstraction
  - **Architecture patterns**
    - Describes structural organisation of application
    - Predefines required subsystems and their responsibilities
    - Specifies rules and guidelines for organising relationships between subsystems
  - **Design patterns**
    - Suggests how to refine subsystems or components within a system
    - Describes desirable relationships between components
    - Describes a known good solution to a commonly occurring design problem in a particular context
  - **Implementation patterns**
    - Low-level patterns of implementation on a specific platform
    - Describes how to implement designs on a specific system
    - May be language-specific

UNIVERSITY of HULL

Patterns for Distributed Systems

These are the key patterns we'll be concentrating on for distributed systems:

- Message-based patterns
  - Help to integrate the components of a distributed system
  - Specific for distributed systems
- Software design patterns
  - General design patterns
  - Useful for distributed systems
  - Support and complement message-based patterns

Message-based patterns

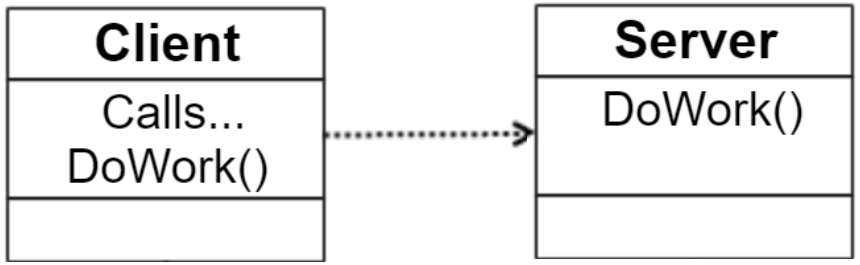
We are going to look at a few basic message-based patterns today

- Point-to-point connections
- Broker
- Message bus

Patterns are EXTREMELY useful in deciding on the core design principles your solution will follow – they are only a pattern though, implementation is still up to you and will be different in different use cases.

I would advise that you continue to research patterns for all software ‘problems’.

Point-to-point pattern

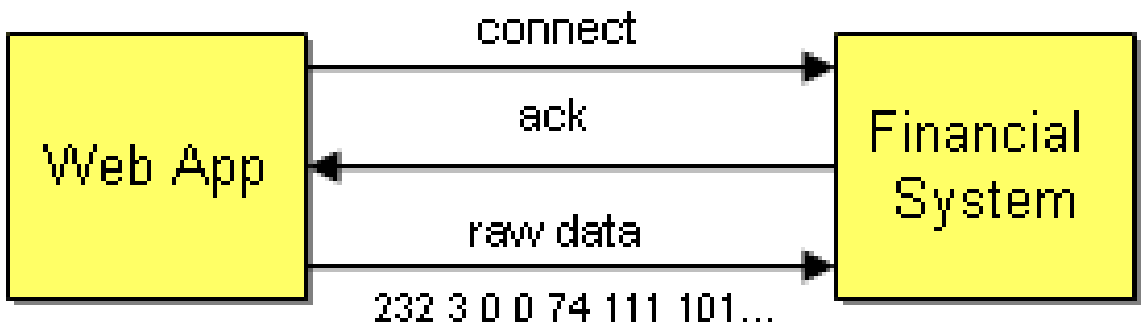


- Simplest pattern
  - One receiver per message
  - No routing
  - Basic client-server

- Connection is managed at the lowest level
- Client must know what the server can do

- Usually some form of Remote Procedure Call

Point-to-point pattern



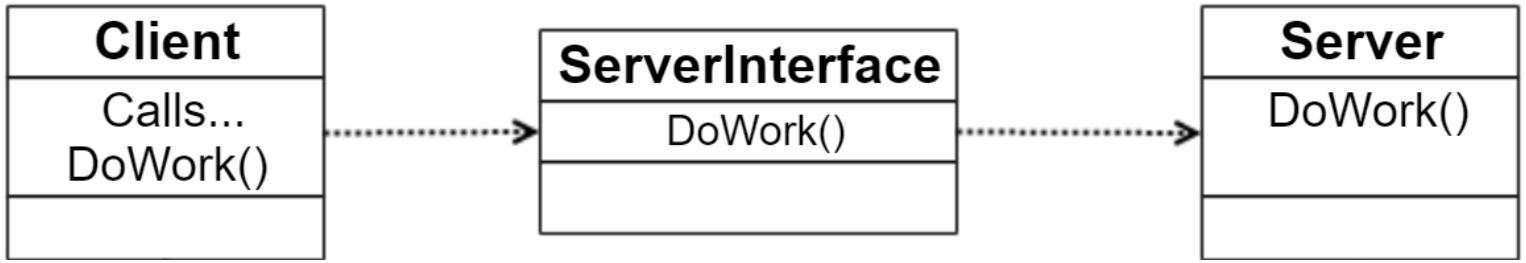
- What are the problems with this pattern?
  - Sender must know receiver's location
  - Communicating with many receivers is a logistical headache
    - Cost of update can be high
  - May involve duplication of effort
    - Each node may require data transformation
  - Both must be available at the same time
  - Client must know what the server can do (hardcoded)

UNIVERSITY of HULL

Using an Interface

- The Broker pattern, and many other Service Oriented Architectures (SOAs) use an interface.
- This means that the client can make calls to a known interface
  - The interface advertises functionality but without implementation details
- The client now isn't reliant on what the server itself does under the hood
  - So the implementation of the server can be changed at any time.

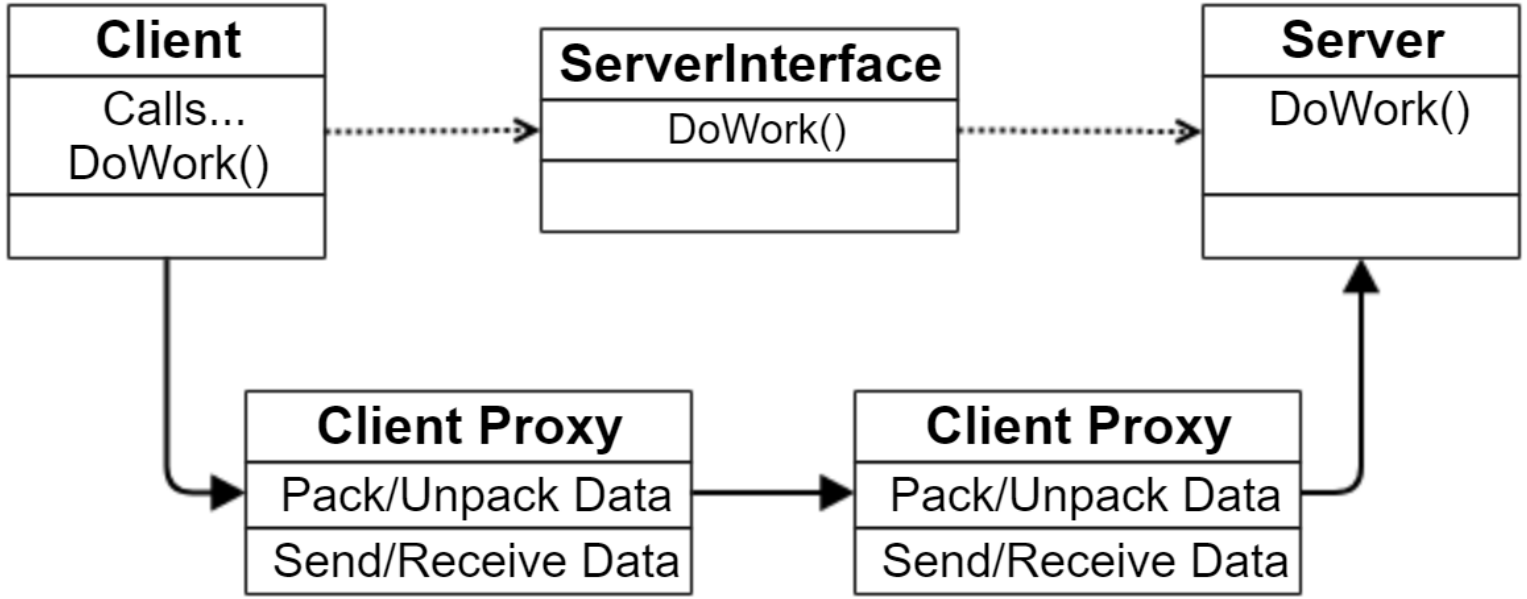
- Usually the interface is advertised to the client in some way.
  - This *could* be at runtime, which means no need to hardcode the remote calls.



UNIVERSITY of HULL

Using a Proxy

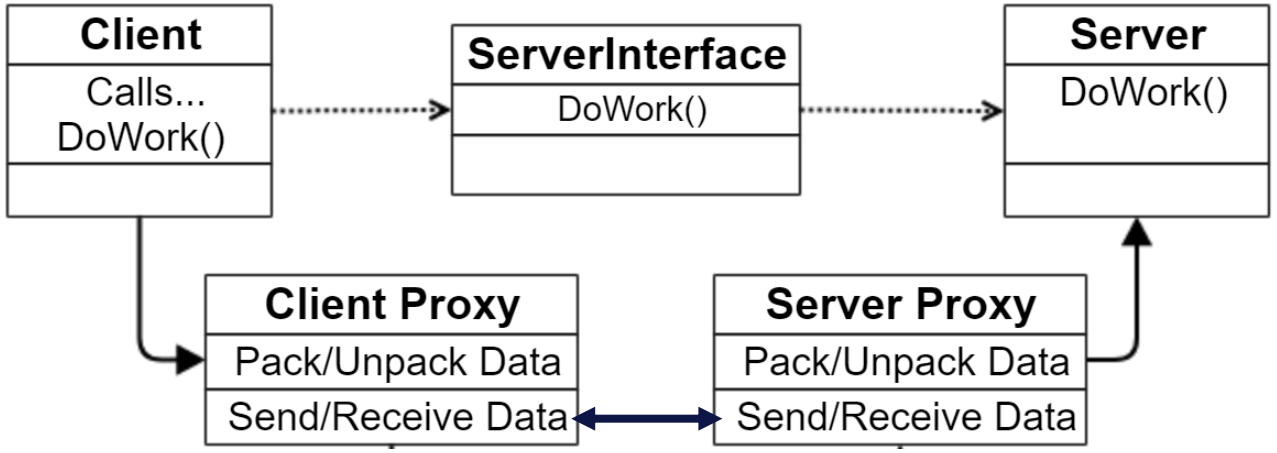
- Proxies manage all of the message sending/receiving/packing/unpacking/security/etc.
- Client understands the server interface but REAL data is sent through the proxy
- This helps improve loose coupling – the client no longer needs to manage the lower-level data transmission tasks.



UNIVERSITY of HULL

Using a Proxy

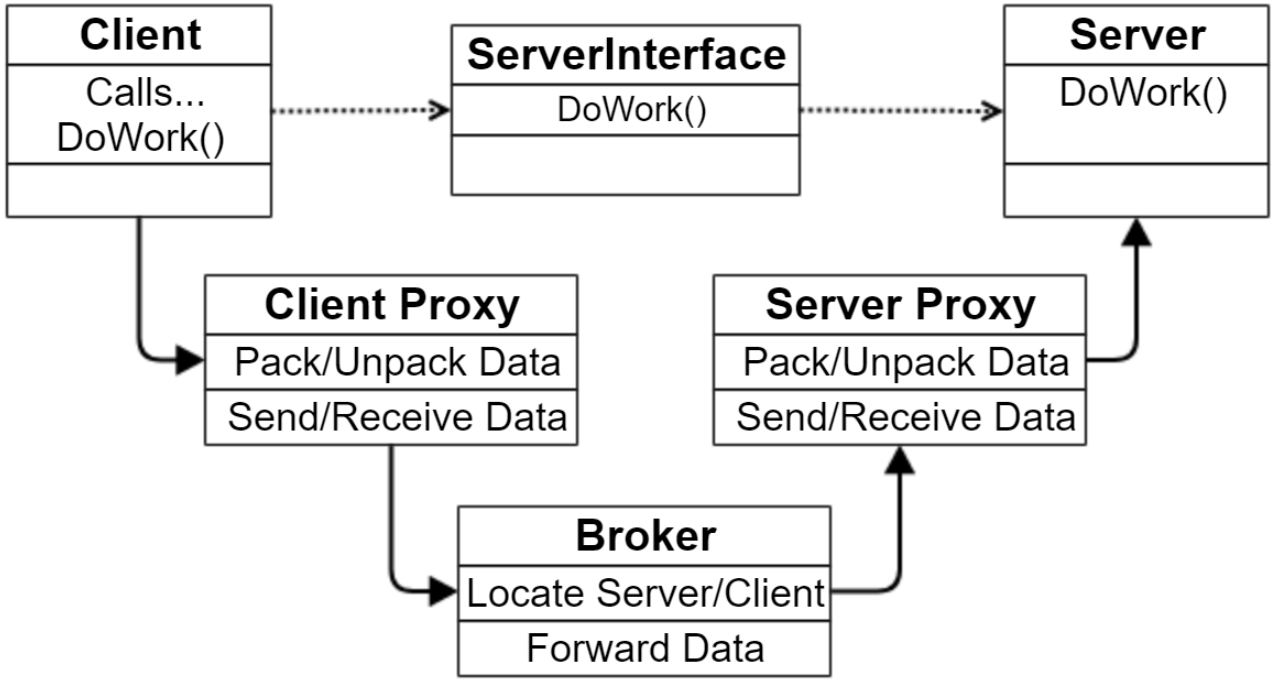
- But there is still a problem… What is the problem here?
  - The client still needs to know where the server is before it can call it
  - Application developers should not have to learn about the details of cross-process remote communications.
  - To maintain flexibility, developers must avoid hard-coding the location of specific components



UNIVERSITY of HULL

Broker

- The Broker manages the location of the client/server and acts as a messenger between the two.
- To the Client and the Server, calls are now routed properly without having to know and store locations.



UNIVERSITY of HULL

Broker Pattern

- Hides the implementation details of remote service invocation by encapsulating them into a layer other than the business component itself
- Provides an interface to the client that allows it to invoke methods just as it would invoke any local interface
- Methods inside the client interface trigger services to be performed on remote objects…

  …or remote services
- Broker may determine location of services…

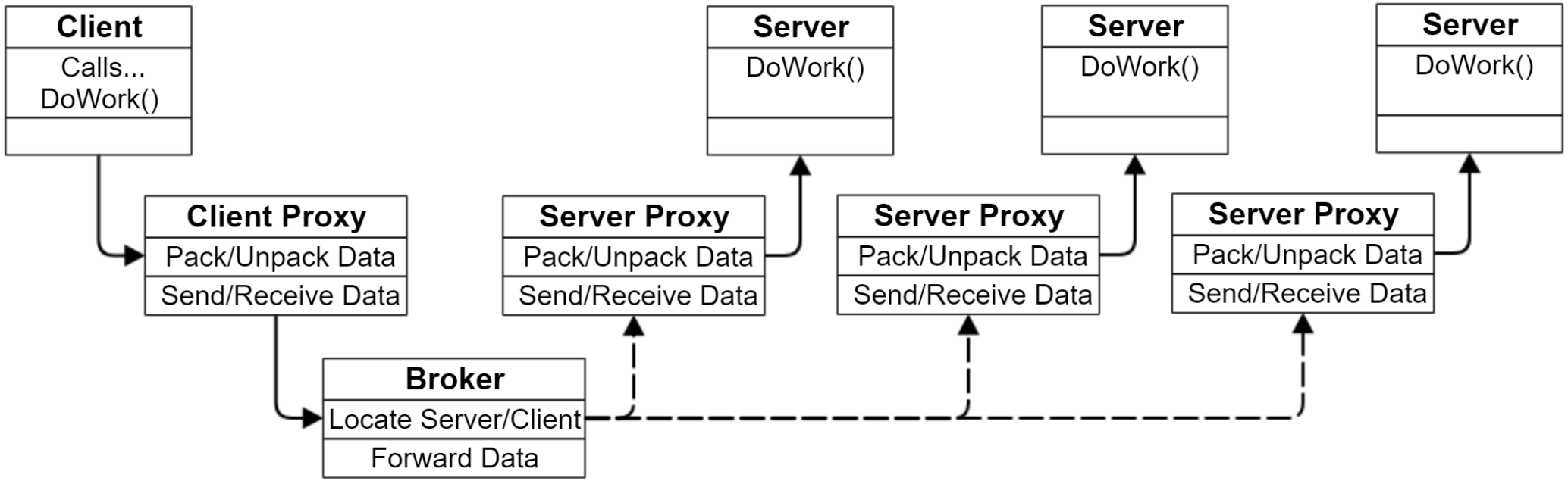
  …client doesn't need to know

  …developer doesn't need to hard-code

UNIVERSITY of HULL

Broker pattern

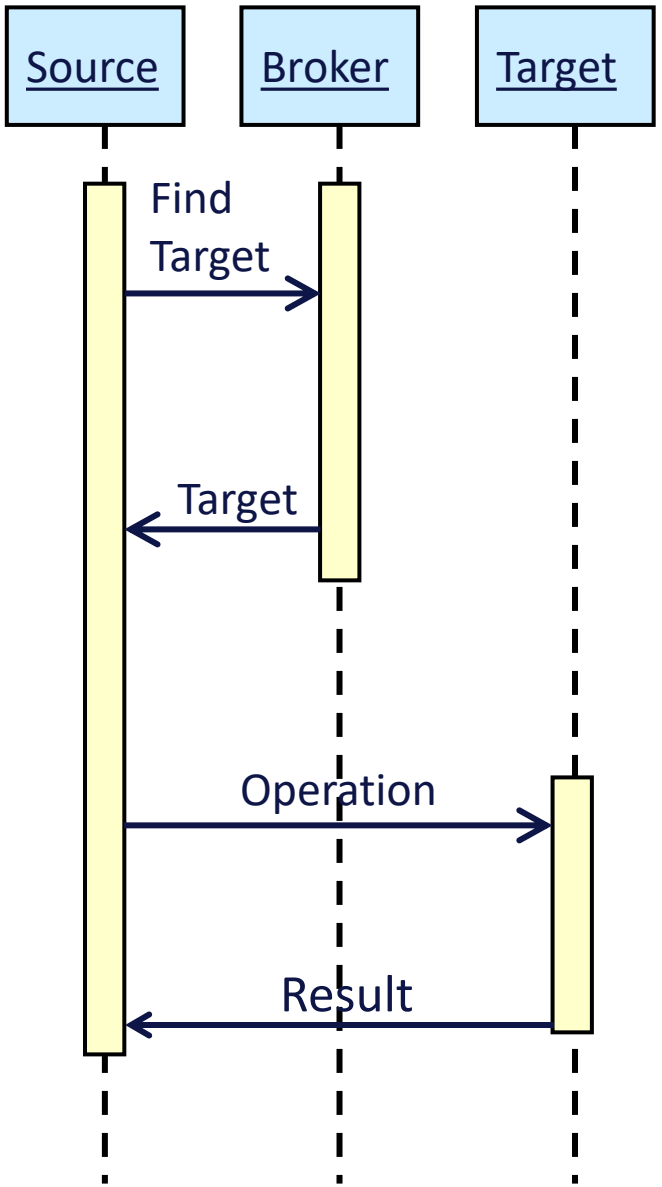
- Required services may be anywhere (and could be multiple different servers)
- Broker acts as intermediary
- Broker maintains a directory
- Client uses service interface
- Broker provides details to proxies for connection
- Servers inform broker of their existence



Direct broker

- Broker establishes communication between endpoints
- Once communication is established, system is point-to-point
- Addresses and connections are centrally managed

- Used by .NET remoting
- Example: naming services
  - Active Directory
  - DNS

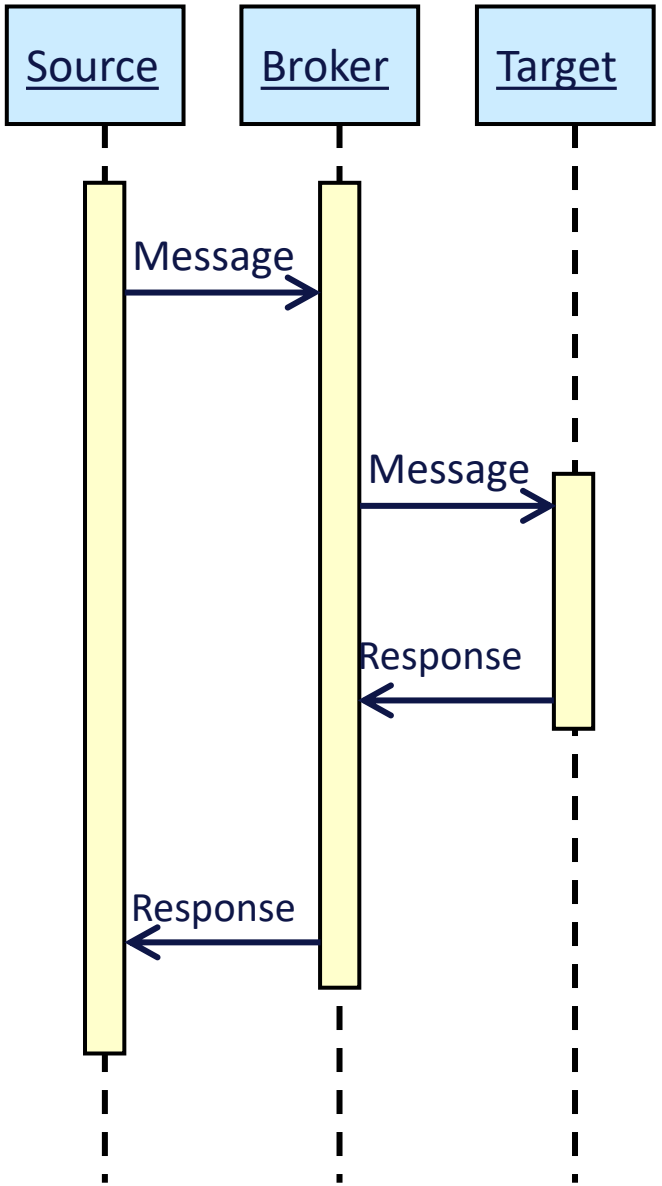


Indirect broker

- All communication is with broker
- Broker passes messages to recipients
- Addresses and connections are centrally managed

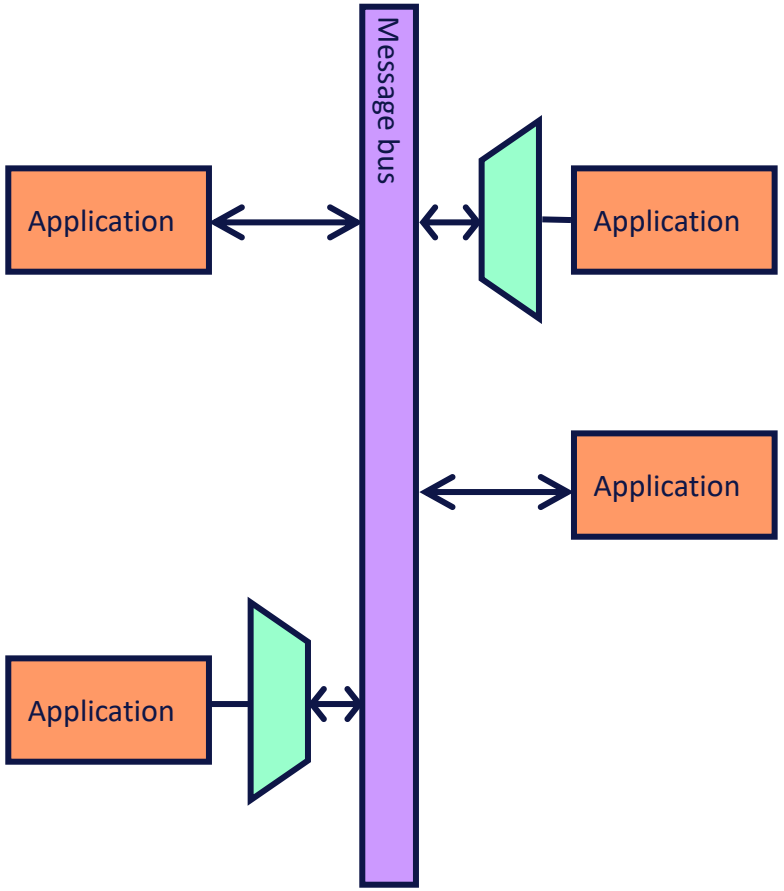
- Not as efficient as direct broker
- Single point of failure
- Single bandwidth bottleneck

- **Q:** In what situation is an indirect broker useful?
  - Brokers can add additional protections, e.g. a firewall



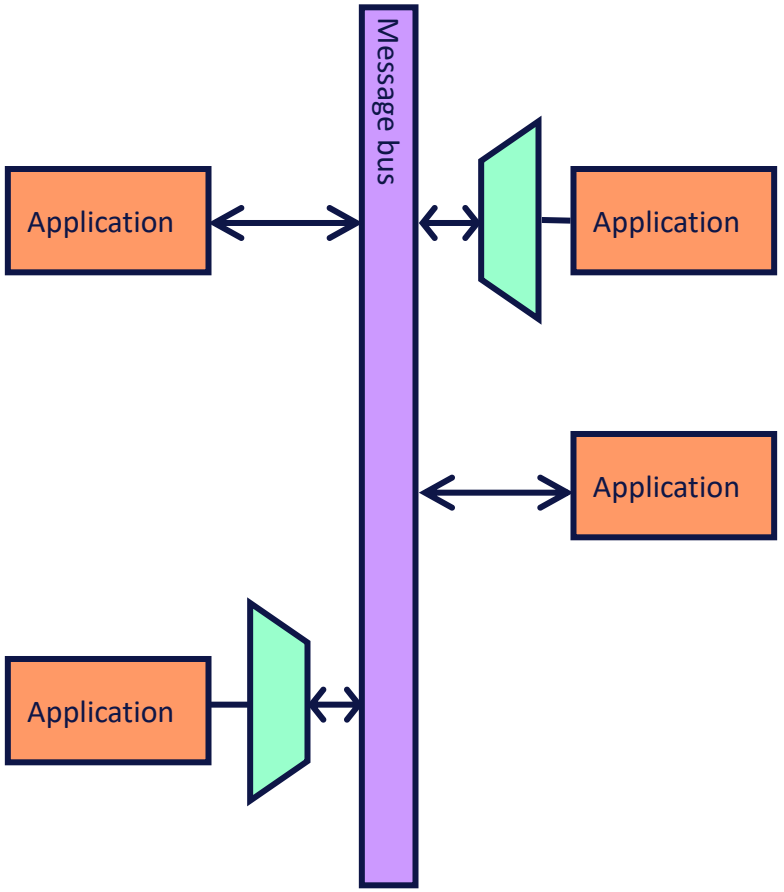
Message Bus

- The *Message bus* pattern defines a standard way of moving data packets from a target system to an intermediary store and transport
- ***May*** provide routing for messages



Message bus

- Applications use message bus middleware to communicate
  - Directly to message bus
  - Indirectly via adapters
- Messages may contain target information
  - May contain routing
  - May be a routing agent in message bus
- Message bus acts as universal connector
- Messages must use a common, standardised, data model
  - Although this could be very basic



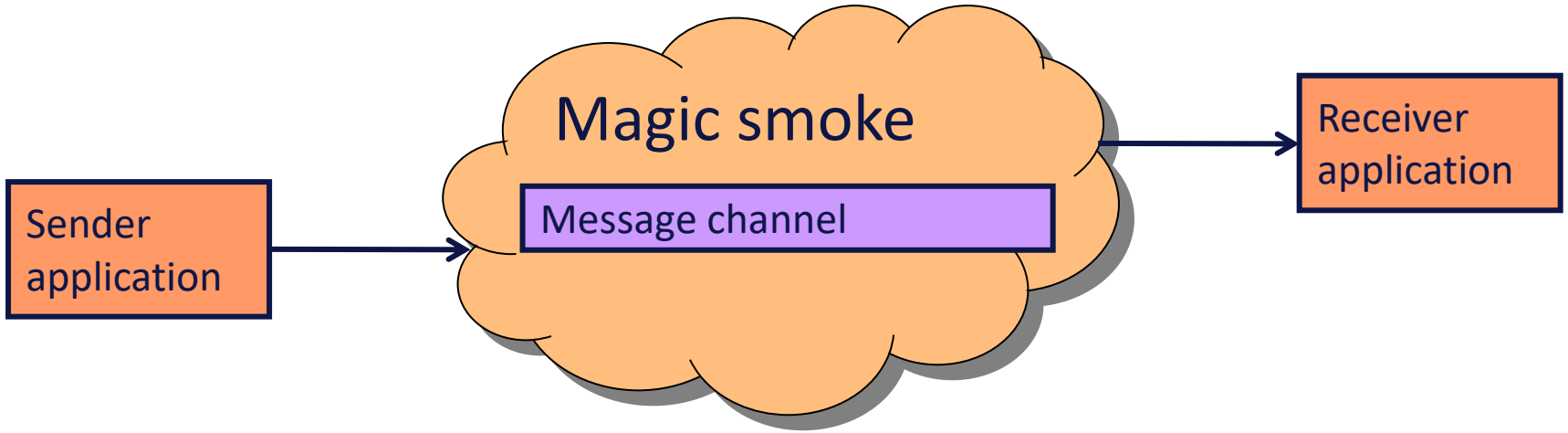
Message bus features

- Common communication infrastructure
  - Cross-platform, cross-language
  - Universal adapter for messages
  - May include routing facilities
    - Message may contain info on who it's for (message bus can do routing if required)
    - Or endpoints may keep track of what is for them (endpoints listen for messages on unrouted bus)
- Adapters
  - Each (application/system/protocol) needs a specific message adapter
  - Can be event generators or data transforms
  - Many filtering operations can be done on messages
- Common command structure
  - Message bus can carry instructions as well as data

UNIVERSITY of HULL

Message channels

- How is Message Bus implemented?



Applications communicate by use of 'Message Channels' which are implemented in "some way"

- Message channel carries specific message to specific destinations OR is common
- Applications may have multiple channels
- Sender puts message on specific channel – receiver retrieves from specific channel
  - Something like streams or pipes

UNIVERSITY of HULL

Message Bus Queueing

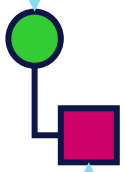
- Generally gives us the option of putting a message into a queue whilst the recipient is not available.
  - Currently “down”
  - Overloaded
- Also may give options for multiple instances of the same recipient software to take from a queue so as to spread the load across devices
- Individual queues can be created where the message bus uses routing

- There are lots of message bus with queue options, e.g.
  - RabbitMQ (open-source)
  - Amazon Simple Queue Service (Cloud)

UNIVERSITY of HULL

Messages

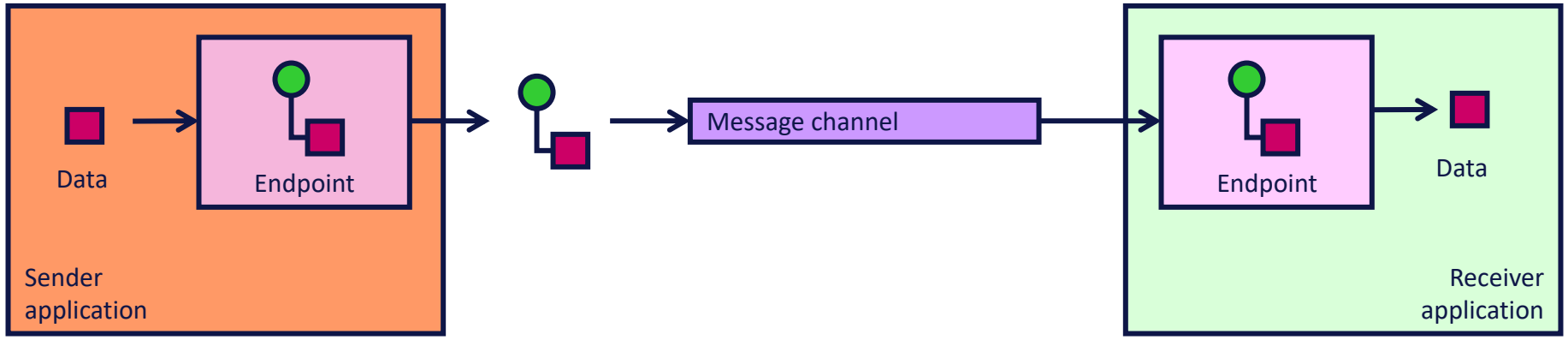
- Header
  - Contains information about the message
    - Destination
    - Source
    - Type
      - DocumentMessage, CommandMessage…
    - Sequence number
  - May be modified by message handling/routing systems
- Body
  - Payload information
  - Generally transmitted verbatim



UNIVERSITY of HULL

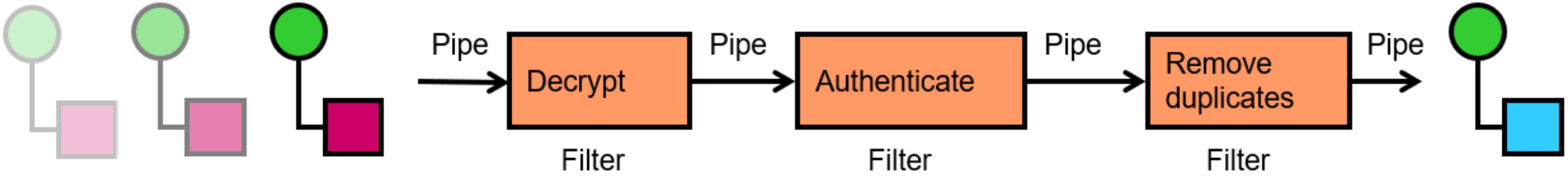
Endpoints

- Sender and receiver are different applications
- May be on different OS or platform
- Endpoint converts data to message, message to data
- Endpoint is channel-specific (so there could be more than one endpoint)
- Manages the lower layers of the protocol so the application itself doesn't need to be concerned with the intricacies of messaging



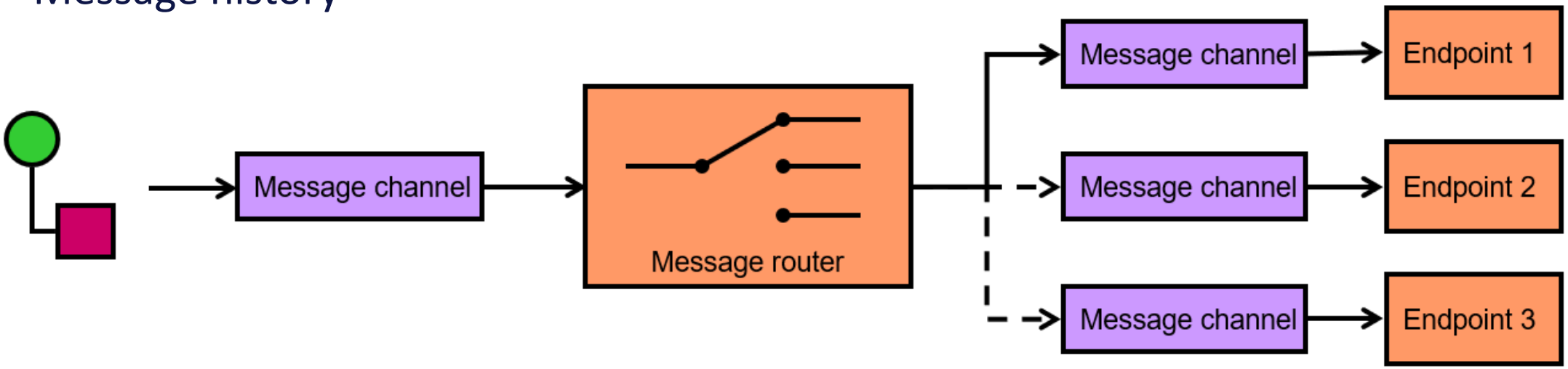
Pipes and filters

- Pipes take messages from one point to another
  - Can be a message channel (if resources permit)
  - In-memory queue may be more efficient if co-located
- Filters (selectively) process messages
  - Performing specific operations
  - May modify message header or body
- Allows common work to be done in the pipe rather than in the application
  - Meaning that the application is only focused on the business logic



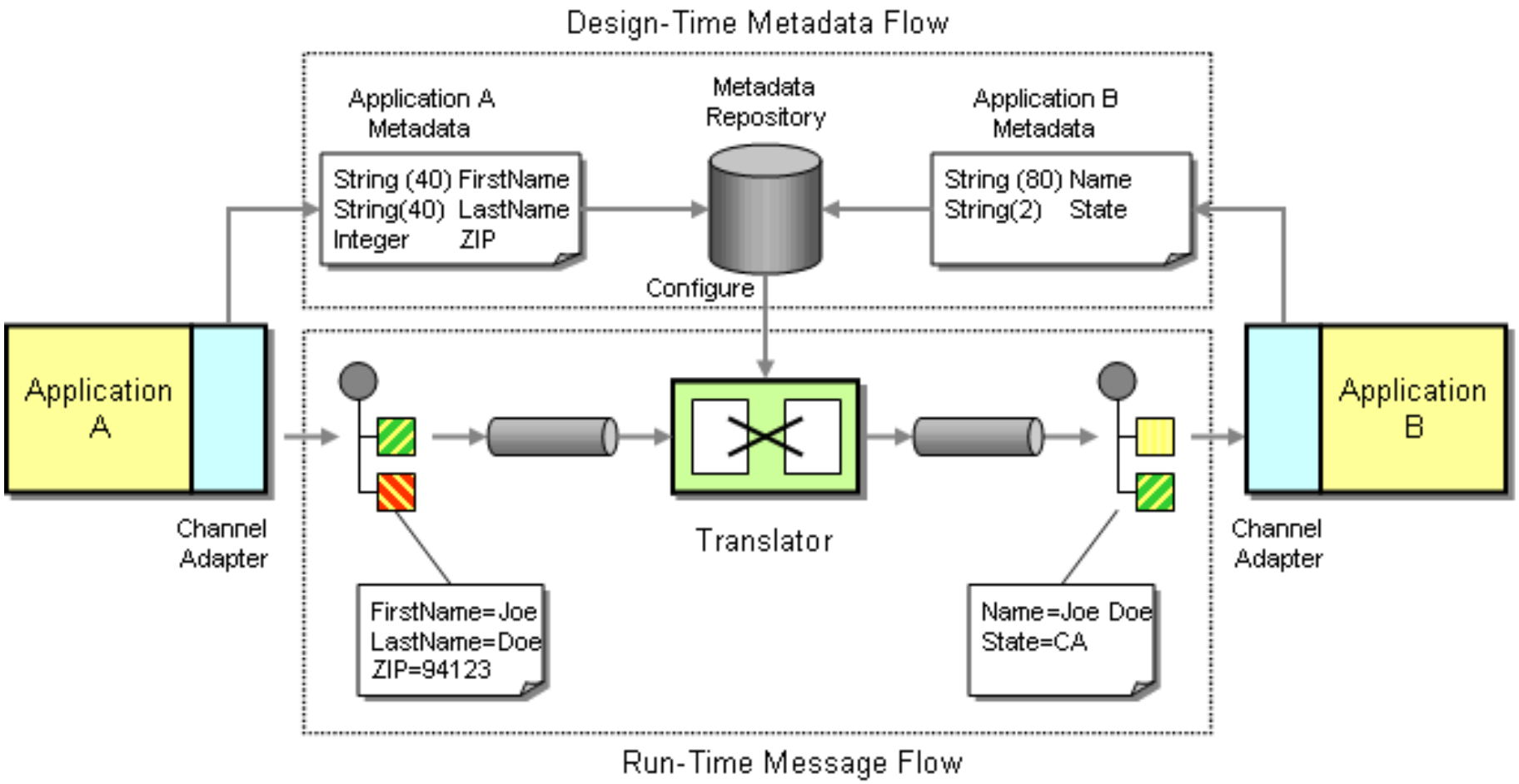
Routing

- Decouple individual processing steps so that messages can be passed to different filters or endpoints depending on a set of conditions
- Might depend on
  - Message content
  - Message type
  - Message header
  - Message history



Transformations

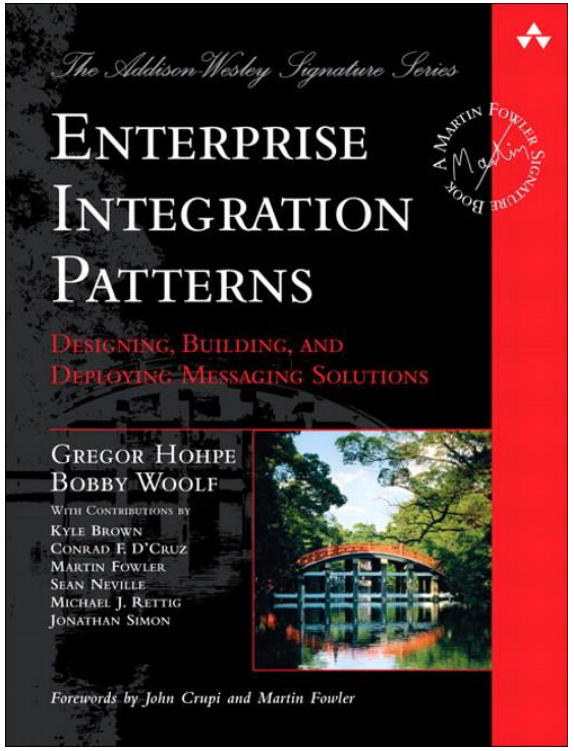
- Filters may transform messages (can be a two-way transformation)
- Change data format and object model based on prior knowledge (metadata)



Book

Enterprise Integration Patterns:
Designing, Building,
and Deploying Messaging Solutions

Gregor Hohpe, Bobby Woolf



Free resources:

https://www.enterpriseintegrationpatterns.com/patterns/messaging/

https://www.enterpriseintegrationpatterns.com/ramblings/eip1_examples_updated.html

https://www.enterpriseintegrationpatterns.com/patterns/conversation/

UNIVERSITY of HULL

Reflection

We've looked at:

- Architectural Pattern Types
- Message-based Patterns
- Messaging
- Pipelines
- Transformations

In the next session we will continue to look at Design Patterns, explicitly Software Design Patterns

UNIVERSITY of HULL